

VERIFACT- AI Based Deepfake Detection System

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Mahi Agrawal

EN23CS301579

Mahimn Geete

EN23CS301581

Manya Pandey

EN23CS301598

Under the Guidance of
Prof. Diptanshu Pandya



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

VERIFACT- AI Based Deepfake Detection System

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Mahi Agrawal

EN23CS301579

Mahimn Geete

EN23CS301581

Manya Pandey

EN23CS301598

Under the Guidance of
Prof. Diptanshu Pandya



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work “**VERIFACT- AI Based Deepfake Detection System**” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

I hereby declare that the project entitled “**VERIFACT- AI Based Deepfake Detection System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Computer Science and Engineering’ completed under the supervision of **Prof. Diptanshu Pandya, Assistant Professor, Department Computer Science and Engineering**, Faculty of Engineering, Medi-Caps University Indore is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

**Mahi Agrawal
Mahimn Geete
Manya Pandey**

Certificate

I, **Prof. Diptanshu Pandya** certify that the project entitled “**VERIFACT- AI Based Deepfake Detection System** ” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Mahi Agrawal , Mahimn Geete, Manya Pandey** is the record carried out by them under my guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Diptanshu Pandya

Computer Science and Engineering

Medi-Caps University, Indore

Dr. Kailash Chandra Bandhu

Head of the Department

Computer Science & Engineering

Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Associate Dean (Computing and Allied Branch), Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Mahi Agrawal

Mahimn Geete

Manya Pandey

B.Tech. III Year (A)

Department of Computer Science & Engineering

Faculty of Engineering

Medi-Caps University, Indore

Abstract

The rapid proliferation of deepfake technology, powered by advancements in artificial intelligence and deep learning, has raised significant concerns regarding misinformation, identity fraud, and digital security. To address these challenges, this project presents *VERIFACT – AI-Based Deepfake Detection System*, a robust and intelligent solution designed to identify manipulated multimedia content with high reliability.

The system leverages deep learning techniques, particularly convolutional neural networks (CNNs), to analyze visual artifacts and facial inconsistencies present in images and videos. It performs multiple stages of processing, including data collection, preprocessing, feature extraction, and classification. The model is trained on a diverse dataset containing both authentic and deepfake samples, enabling it to learn subtle discrepancies such as abnormal facial expressions, lighting inconsistencies, and pixel-level distortions.

The proposed system architecture follows a modular approach, consisting of a user interface for media upload, a backend processing unit for detection, and a results module that provides authenticity predictions. The system is designed to be scalable, efficient, and user-friendly, allowing users to verify digital content in real time.

VERIFACT not only enhances the detection of synthetic media but also contributes to the broader goal of maintaining trust in digital ecosystems. Experimental evaluation demonstrates that the system achieves promising accuracy in distinguishing real and fake content, making it a valuable tool in combating the misuse of AI-generated media.

Keywords:

Deepfake Detection, Artificial Intelligence, Machine Learning, Deep Learning, Convolutional Neural Networks (CNN), Image and Video Analysis, Facial Feature Extraction, Media Forensics, Digital Security, Misinformation Detection, Pattern Recognition, Authenticity Verification

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	viii
	List of tables	ix
	Abbreviations	x
	Notations & Symbols	xi
Chapter 1	Introduction (Whichever is applicable)	
	1.1 Introduction	1
	1.2 Literature Review	1
	1.3 Objectives	2
	1.4 Significance	2
	1.5 Research Design	2
	1.6 Source of Data	2
	1.7 Chapter Scheme	3
Chapter 2	REQUIREMENTS SPECIFICATION	
	2.1 User Characteristics	4
	2.2 Functional Requirements	4
	2.3 Dependencies	4
	2.4 Performance Requirements	5
	2.5 Hardware Requirements	5
	2.6 Constraints & Assumptions	5
Chapter 3	DESIGN (Whichever is applicable)	
	3.1 Algorithm (if Applicable)	6
	3.2 Function Oriented Design for procedural approach	6
	3.3 System Design (Whichever is applicable)	6
	3.3.1 Data Flow Diagrams (Level 0,Level1)	7
	3.3.2 Activity Diagram	7
	3.3.3 Flow Chart	9
	3.3.4 Class Diagram	10
	3.3.5 ER Diagram	10

	3.3.6 Sequence diagram	11
	3.4 Database Design	12
	3.4.1 Logical Database Design	12
	3.4.2 Physical Database Design	12
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	13
	4.2 Testing Techniques and Test Plans (According to project)	16
	4.3 Installation Instructions	18
	4.4 End User Instructions	19
Chapter 5	Results and Discussions	
	5.1 User Interface Representation (of Respective Project)	21
	5.2 Brief Description of Various Modules of the system	21
	5.3 Snapshots of system with brief detail of each	22
	5.4 Back Ends Representation (Database to be used)	25
	5.5 Snapshots of Database Tables with brief description	26
Chapter 6	Summary and Conclusions	27
Chapter 7	Future scope	28
	Appendix	29-30
	Bibliography	31
	List of Publications (If any)	
	Reprints of publications(If any)	

List of Figures

- **Fig 3.3.1:** Data Flow Diagram (DFD)
- **Fig 3.3.2:** Activity Diagram of VERIFACT System
- **Fig 3.3.3:** Model Training Workflow
- **Fig 3.3.4:** VERIFACT: Deepfake Detection System UML Class Diagram
- **Fig 3.3.5:** ER Diagram: AI Deepfake Detection System (VERIFACT) – SQLite Schema
- **Fig 3.3.6:** Sequence Diagram of VERIFACT System
- **Fig 5.3.1:** Home Page
- **Fig 5.3.2:** Upload Interface
- **Fig 5.3.3:** Media Preview Screen
- **Fig 5.3.4:** Detection Result Screen
- **Fig 5.5.1:** Users Table Snapshot
- **Fig 5.5.2:** Detection Results Table Snapshot

list of Tables

Table No.	Table Name	Page No.
Table 1.1	List of Abbreviations and Acronyms	x
Table 1.2	Notations and Mathematical Symbols	xi
Table 4.2	System Test Plan and Results	17
Table 7.1	Comprehensive List of Test Cases (Appendix C)	30

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
UI	User Interface
DFD	Data Flow Diagram
UML	Unified Modeling Language
GPU	Graphics Processing Unit
API	Application Programming Interface

Table 1.1

Notations & Symbols

Symbol / Notation	Meaning
X	Input Data (Image/Video)
Y	Output Prediction (Real/Fake)
$f(x)$	Model Function
w	Weights of the Model
b	Bias Term
$\hat{y}$	Predicted Output
L	Loss Function
Σ	Summation
$\rightarrow$	Data Flow Direction
%	Accuracy Percentage

Table 1.2

Chapter 1

Introduction

1.1 Background of the Study

In recent years, the rapid advancement of Artificial Intelligence (AI) and Deep Learning (DL) has significantly transformed the way digital content is created and consumed. One of the most notable developments in this domain is deepfake technology, which uses sophisticated algorithms to generate highly realistic fake images, videos, and audio. These manipulations are often indistinguishable from real content, making them a major concern in today's digital world.

Deepfake technology primarily relies on deep learning models such as Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs). While it has beneficial applications in areas like entertainment, virtual reality, and education, it is also widely misused for spreading misinformation, identity theft, cyber fraud, and fake news.

To address these challenges, this project introduces *VERIFACT – AI-Based Deepfake Detection System*, which aims to automatically detect manipulated media content using advanced machine learning techniques. The system analyzes visual patterns, facial inconsistencies, and pixel-level distortions to classify media as real or fake, thereby contributing to digital security and trust.

1.2 Literature Review

Several researchers have explored methods for detecting deepfake content using machine learning and deep learning techniques.

Early approaches relied on traditional image processing and manual inspection methods, which were time-consuming and less accurate. With the advancement of deep learning, more efficient techniques have been developed.

Studies have shown that Convolutional Neural Networks (CNNs) are highly effective in detecting spatial inconsistencies in images. Similarly, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks have been used for analyzing temporal inconsistencies in videos.

Recent research focuses on detecting artifacts such as unnatural facial movements, irregular blinking patterns, lighting mismatches, and pixel-level distortions. Some models also use frequency domain analysis and attention mechanisms to improve detection accuracy.

Despite these advancements, existing systems still face challenges in detecting highly realistic deepfakes and maintaining accuracy across diverse datasets. Therefore, there is a need for more robust and efficient systems like VERIFACT.

1.3 Objectives

The main objectives of this project are:

- To develop an AI-based system for detecting deepfake images .
- To implement deep learning techniques such as CNN for feature extraction and classification
- To analyze facial features and detect inconsistencies in digital media
- To design a user-friendly interface for uploading and analyzing media files
- To achieve high accuracy and efficiency in detecting manipulated content
- To enhance digital security and reduce the spread of misinformation

1.4 Significance

The significance of this project lies in its ability to address critical issues related to digital media authenticity and cybersecurity.

- Helps in detecting fake media and preventing misinformation
- Protects individuals and organizations from identity fraud
- Enhances trust in digital platforms and online communication
- Useful for journalists, law enforcement agencies, and social media platforms
- Contributes to research and development in artificial intelligence and cybersecurity

Thus, VERIFACT plays an important role in ensuring the credibility and reliability of digital content.

1.5 Research Design

The research design of this project follows a systematic and structured approach:

1. Data Collection–Gathering datasets containing real and deepfake images/videos
2. Data Preprocessing – Cleaning, resizing, and normalizing the data
3. Feature Extraction – Extracting important visual features using CNN
4. Model Training – Training the deep learning model using labeled data
5. Model Evaluation – Testing the model using performance metrics like accuracy, precision, and recall
6. Deployment – Integrating the model into a user interface for real-time detection

1.6 Source of Data

The data used in this project is collected from publicly available datasets and online sources. These datasets include both real and manipulated media samples required for training and testing the model.

Examples of data sources may include:

- Public deepfake datasets (e.g., FaceForensics++, DeepFake Detection Challenge dataset)
- Open-source image repositories
- Self-collected or preprocessed datasets for experimentation

The dataset is divided into training and testing sets to evaluate model performance effectively.

1.7 Chapter Scheme

The report is organized into the following chapters:

- Chapter 1: Introduction – Provides background, literature review, objectives, significance, research design, and data sources of the project.
- Chapter 2: System Analysis – Describes system requirements, feasibility study, and problem analysis.
- Chapter 3: System Design – Includes architecture diagrams, DFD, and UML diagrams.
- Chapter 4: Implementation – Explains tools, technologies, and modules used in the project.
- Chapter 5: Results and Discussion – Presents output results, performance analysis, and evaluation.
- Chapter 6: Conclusion and Future Scope – Summarizes the project and suggests future improvements.

Chapter 2: Requirements Specifications

2.1 User Characteristics

The VERIFACT system is designed to be used by a diverse group of users with varying levels of technical expertise. The primary users include general individuals, students, researchers, and professionals who need to verify the authenticity of digital media.

General users are expected to have basic knowledge of computer operations such as uploading files and interpreting results. No specialized technical skills are required for operating the system. Researchers and students may have additional knowledge of machine learning concepts and may use the system for academic or experimental purposes. Media professionals and organizations may utilize the system for verifying the credibility of images and videos before publication.

Overall, the system is designed to be simple, intuitive, and accessible to all types of users.

2.2 Functional Requirements

The system performs several essential functions to ensure accurate detection of deepfake content. It allows users to upload image through a user-friendly interface. Once the input is received, the system preprocesses the data by resizing, normalizing, and preparing it for analysis.

The system then extracts relevant features using deep learning techniques and processes the input through a trained model. Based on the analysis, the system classifies the media as either real or fake. The output is displayed clearly to the user along with the prediction result.

Additionally, the system is capable of handling invalid inputs or unsupported file formats and ensures smooth operation during the detection process.

2.3 Dependencies

The proper functioning of the VERIFACT system depends on several software, hardware, and data-related components. The system requires a Python-based environment along with essential libraries such as TensorFlow, Keras, OpenCV, NumPy, and Pandas.

From a hardware perspective, the system requires a computer with sufficient processing power and memory. A GPU is recommended for faster training and improved performance but is not mandatory.

The system also depends on the availability of labeled datasets containing both real and deepfake media for training and evaluation purposes. In case of a web-based implementation, an internet connection may be required for data access and deployment.

2.4 Performance Requirements

The system is expected to perform efficiently while maintaining high accuracy in detecting deepfake content. It should provide quick responses to user inputs and ensure minimal delay during processing.

The model should be capable of handling moderate to large datasets and should scale effectively if deployed in a multi-user environment. The system must also maintain reliability and consistency in its predictions across different types of media inputs.

Low latency, high accuracy, and efficient resource utilization are key performance goals of the system.

2.5 Hardware Requirements

The minimum hardware requirements for implementing the VERIFACT system include a system with an Intel i5 processor or equivalent, along with at least 8 GB of RAM to ensure smooth processing. A minimum of 256 GB of storage is recommended to store datasets, models, and related files.

Although not mandatory, the use of a dedicated GPU, such as an NVIDIA graphics card, can significantly improve the training speed and overall performance of the deep learning model. The system should run on a 64-bit operating system.

2.6 Constraints and Assumptions

The system operates under certain constraints and assumptions. One of the primary constraints is that the accuracy of the model heavily depends on the quality and size of the dataset used for training. Additionally, training deep learning models requires significant computational resources, which may not be available in all environments.

The system may face challenges in detecting highly advanced deepfake content that closely mimics real media. Processing large video files may also increase execution time.

It is assumed that users have basic knowledge of using computers and that the input media files are provided in supported formats such as JPEG, PNG, or MP4. It is also assumed that all required software libraries and dependencies are correctly installed and configured before using the system.

Chapter 3: Design

3.1 Algorithm

The VERIFACT system follows a structured algorithm to detect deepfake content efficiently. The algorithm is based on deep learning techniques using Convolutional Neural Networks (CNN) for classification.

Algorithm Steps:

1. Start
2. Accept input media file (image/video) from the user
3. Perform preprocessing (resize, normalize, frame extraction for video)
4. Extract relevant features from the input data
5. Load the trained deep learning model
6. Pass the processed data into the model
7. Analyze features and predict output
8. Classify the input as **Real** or **Fake**
9. Display the result to the user
10. Stop

This algorithm ensures systematic processing and accurate classification of digital media.

3.2 Function Oriented Design for Procedural Approach

The system is designed using a function-oriented approach where the overall system is divided into smaller functional modules. Each module performs a specific task, ensuring modularity and ease of maintenance.

The major functions of the system include:

- **Input Function:** Accepts media files from the user
- **Preprocessing Function:** Cleans and prepares the data for analysis
- **Feature Extraction Function:** Extracts important visual features
- **Classification Function:** Uses trained CNN model to classify input
- **Output Function:** Displays the prediction result

This modular design improves efficiency, scalability, and reusability of the system.

3.3 System Design

The system design defines the overall architecture and workflow of the VERIFACT system. It consists of three major components:

- **User Interface Layer:** Allows users to upload media and view results
- **Processing Layer:** Handles preprocessing, feature extraction, and model prediction
- **Output Layer:** Displays classification results

3.3.1 Data Flow Diagrams (DFD)

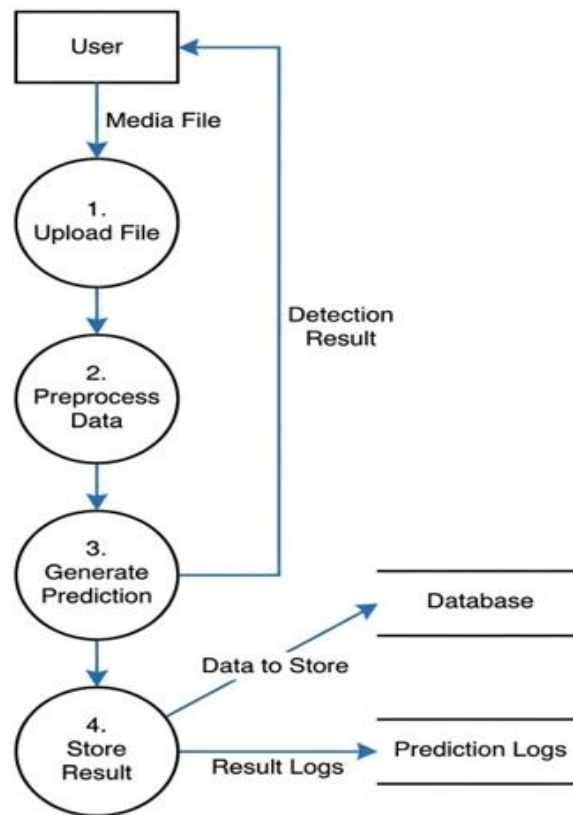


Fig 3.3.1

3.3.2 Activity Diagram

The activity diagram represents the flow of activities in the system:

Start → Upload Media → Preprocess Data → Extract Features → Run Model → Generate Prediction → Display Result → End

This diagram helps in understanding the step-by-step execution flow of the system.

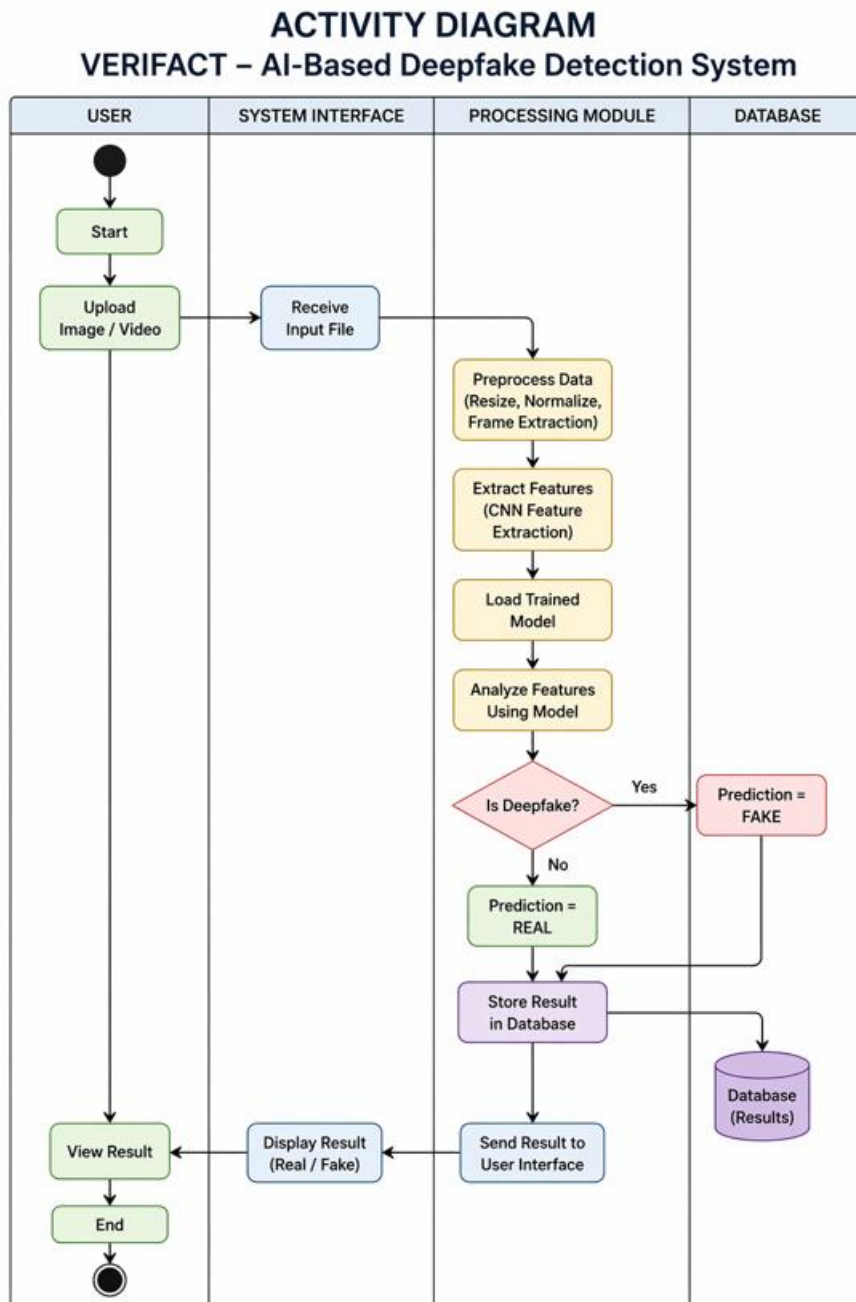


Fig 3.X: Activity Diagram of VERIFACT System

Fig 3.3.2

3.3.3 Flow Chart

A Flow chart shows the step-by-step working of our system from input to output, it shows To show how VERIFACT processes a deepfake detection request end-to-end.

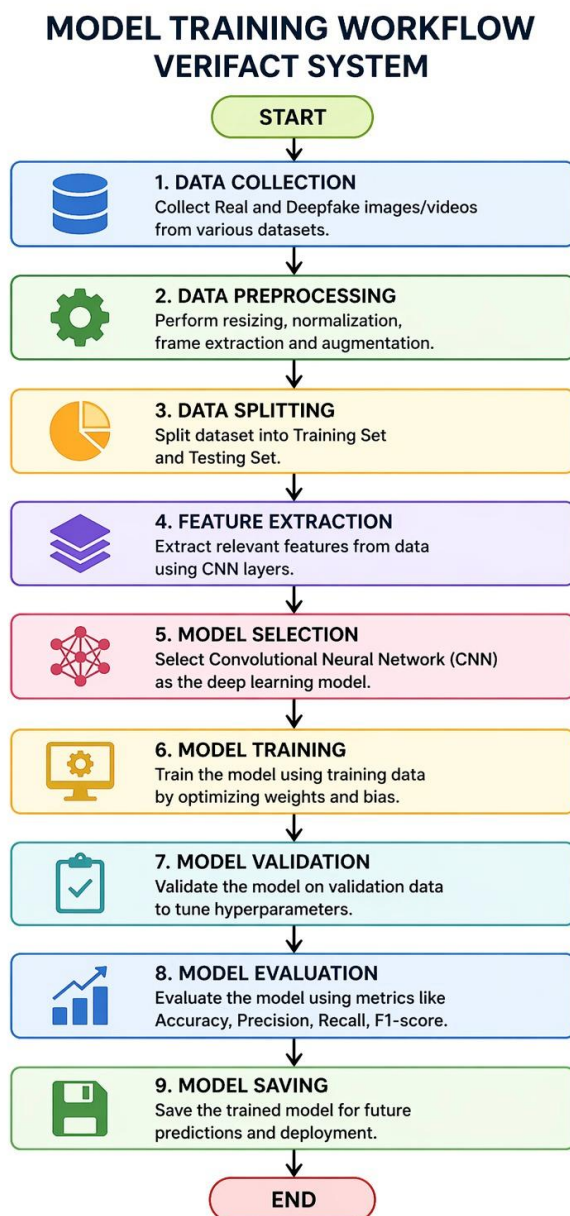


Fig 3.3.3

3.3.4 Class Diagram

A Class Diagram Shows structure of your system using objects/classes and their attributes + functions , it Shows how system components are structured in code.

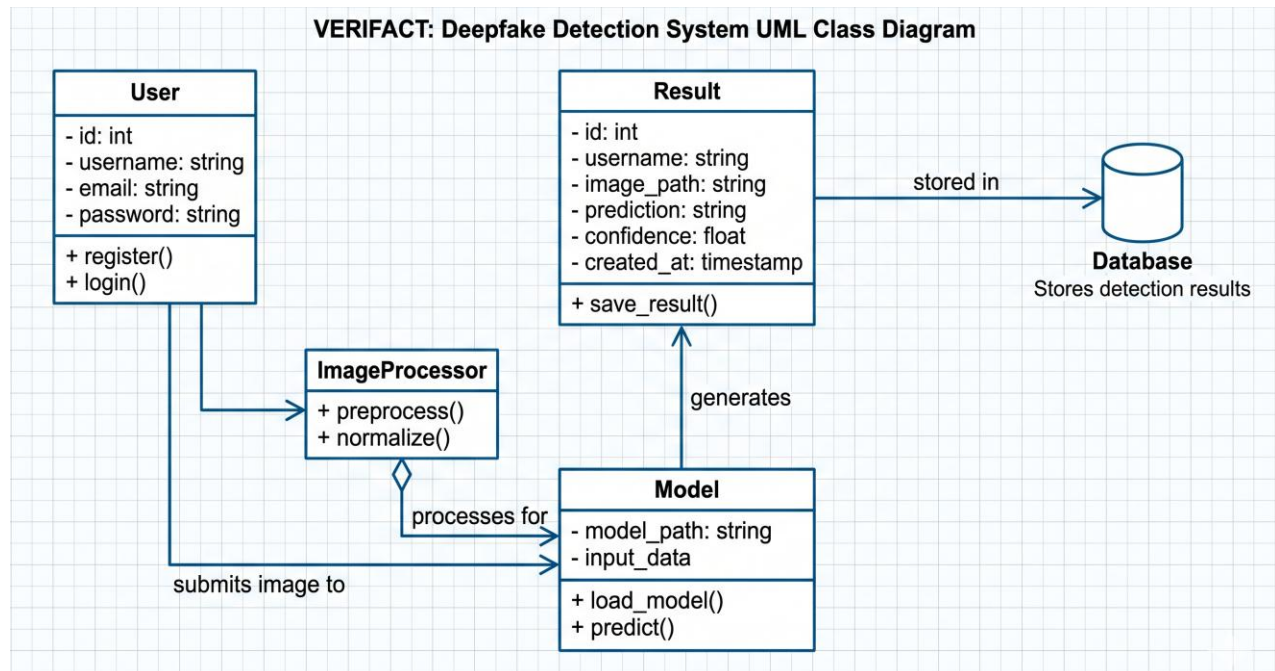


Fig 3.3.4

3.3.5 ER Diagram

An ER diagram shows relationship between database tables. Example One user → can have many results (1 : N relationship)

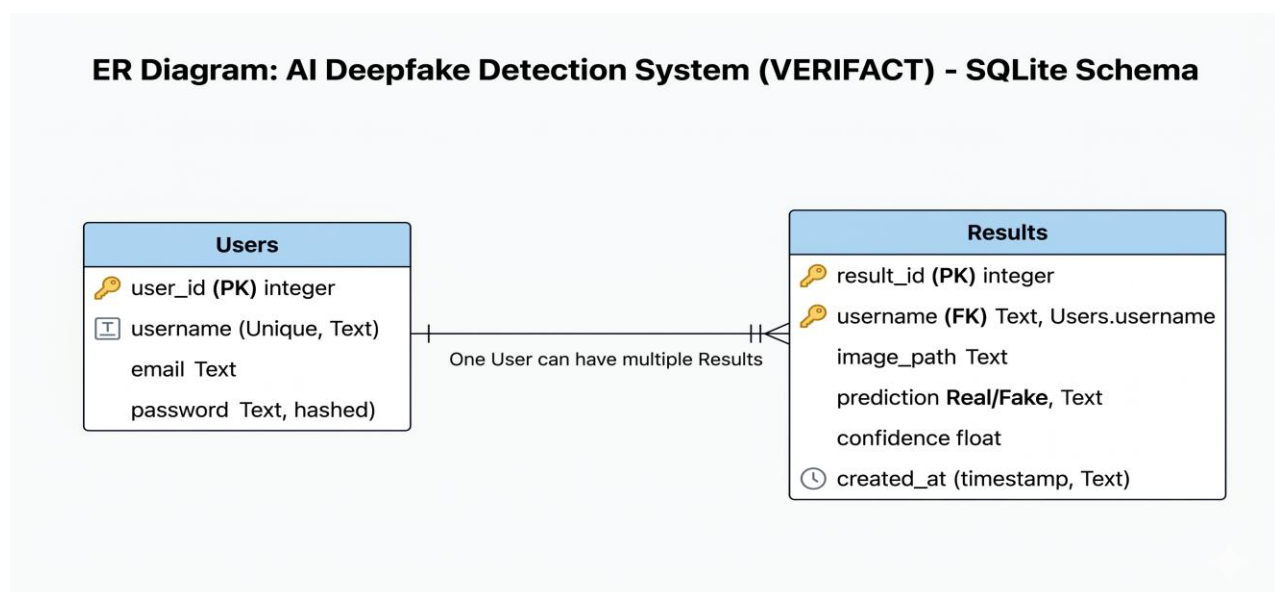


Fig 3.3.5

3.3.6 Sequence Diagram

The sequence diagram shows the interaction between different components over time:

User → UI → Preprocessing Module → Model → Result Module → User

It illustrates how data flows sequentially from input to output.

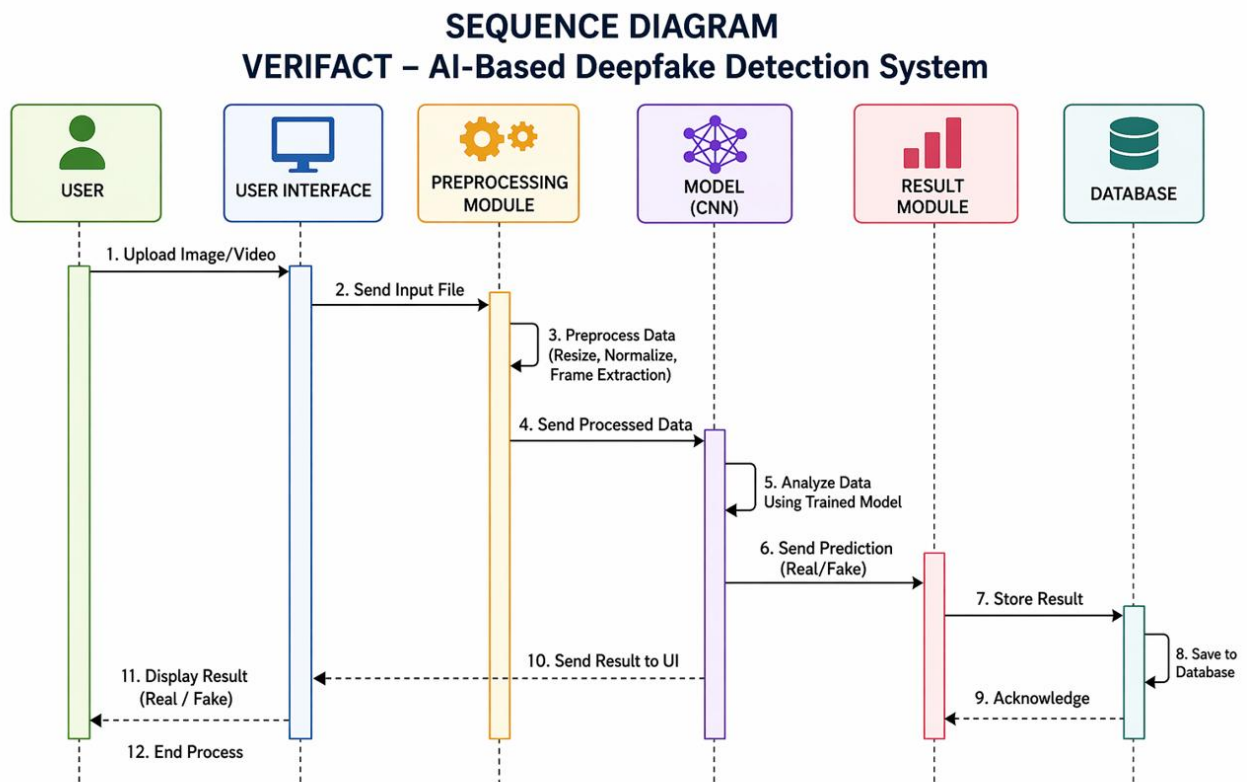


Fig 3.X: Sequence Diagram of VERIFACT System

Fig 3.3.6

3.4 Database Design

The system uses a simple database structure (if implemented) to store user inputs and results.

3.4.1 Logical Database Design

The logical design defines the structure of data in terms of entities and relationships:

- User Table (User_ID, Name, Email, Password)
- Result Table (Result_ID, Username, image_path, Prediction, confidence, created_at)

3.4.2 Physical Database Design

The physical design defines how data is stored in the system:

- Data stored in structured tables
- File paths used to store media
- Lightweight database (e.g., SQLite) or file-based storage

Chapter 4: Implementation, Testing and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies Used for Implementation

The implementation of **VeriFact**, an AI-based Deepfake Detection System, was carried out using a combination of web technologies, machine learning frameworks, and development tools to build an efficient and scalable application.

Programming Languages Used

Python

Python was used as the core programming language for backend development and machine learning implementation due to its extensive support for artificial intelligence and data processing.

Applications:

- Media preprocessing
- Deepfake detection model implementation
- Backend logic handling
- Confidence score generation

JavaScript

JavaScript was used in frontend development to create dynamic and interactive user interfaces.

Uses:

- Handling user actions
- Upload interactions
- Displaying detection results dynamically

HTML and Tailwind CSS

These were used for structuring and styling the user interface.

- HTML for page structure
- Tailwind CSS for responsive and modern UI design

Frameworks and Libraries

React JS

React was used for frontend development to create reusable components and enhance user experience.

Features implemented:

- Upload interface
- Preview section
- Result display module
- Reset and analyze buttons

PyTorch

PyTorch was used as the primary deep learning framework for implementing the deepfake detection model.

Functions:

- Model training
- Feature extraction
- Real/Fake classification

Pillow (PIL)

Pillow (Python Imaging Library) was used for image processing tasks.

Applications:

- Loading and reading images
- Resizing and formatting images
- Basic image preprocessing

Torchvision

Torchvision was used along with PyTorch for image transformations and dataset handling.

Functions:

- Image normalization
- Data augmentation (cropping, scaling, etc.)
- Converting images into tensor format

NumPy

NumPy was used for numerical operations and data manipulation during preprocessing and model computation.

Development Tools / IDEs

Visual Studio Code

Visual Studio Code was the primary IDE used for development.

Uses:

- Frontend coding
- Backend development
- Debugging
- Version control integration

Jupyter Notebook / Google Colab

These tools were used for experimentation and model training.

Applications:

- Dataset experimentation
- Model training
- Accuracy evaluation
- GPU-supported execution

Version Control

Git and GitHub

Git and GitHub were used for version control and project management.

Uses:

- Source code management
- Tracking changes
- Maintaining project repository

4.2 Testing Techniques and Test Plans

To verify correctness, performance, and reliability of VeriFact, several testing techniques were used.

Testing Techniques

1. Unit Testing

Individual modules were tested separately.

Modules tested:

- Media upload module
- File validation module
- Preprocessing module
- Deepfake detection model
- Result display module

2. Integration Testing

Ensured proper communication among:

- React frontend
- Python backend
- ML detection model

Verified:

- File upload → preprocessing
- Preprocessing → model analysis
- Model output → result display

3. System Testing

Complete system was tested as a whole.

Checked:

- Real image detection
- Deepfake image detection
- Video analysis
- Confidence score generation

Examples:

- **REQ-1:** Media upload functionality was tested to ensure successful selection and submission of image files.
- **REQ-2:** Input validation was tested to ensure only supported file formats are accepted.
- **REQ-3:** Image preprocessing was tested, including resizing, normalization, and tensor conversion.

- **REQ-4:** Model loading and initialization were tested to ensure the trained deep learning model loads correctly.
- **REQ-5:** Feature extraction using the deep learning model was tested for correctness.
- **REQ-6:** Prediction module was tested to ensure accurate real/fake classification.
- **REQ-7:** Confidence score generation was tested to display probability values correctly.
- **REQ-8:** Result display interface was tested to ensure proper rendering of output on the frontend.
- **REQ-9:** Reset and re-upload functionality was tested for repeated use of the system.
- **REQ-10:** Error handling was tested for invalid or corrupted input files.
- **REQ-11:** UI responsiveness was tested across different screen sizes and devices.
- **REQ-12:** End-to-end system flow was tested from upload to prediction output.

Test Plan

Test Case	Input	Expected Result	Status
Valid image upload	Real image	Accepted	Pass
Invalid file format	Unsupported file	Rejected	Pass
Deepfake media upload	Fake media	Detected as Fake	Pass
Genuine media upload	Real media	Detected as Real	Pass
Reset button	User click	Interface resets	Pass

Table 4.2.1

Performance Testing

Performance was evaluated using:

- Image response time < 5 seconds
- Accuracy benchmarks using test dataset

Metrics used:

- Accuracy
- Precision
- Recall
- F1-score

Security Testing

Performed to ensure:

- Invalid uploads are blocked
- Malicious files are rejected
- User uploaded data remains secure

4.3 Installation Instructions

System Requirements

Hardware Requirements

Minimum:

- 8 GB RAM
- Intel i5 Processor or equivalent
- 10 GB free disk space

Recommended:

- GPU support for faster model inference

Software Requirements

- Python 3.x
- Node.js
- React environment
- Required Python libraries
- Web browser

Installation Steps

Step 1: Clone the Repository

```
git clone https://github.com/project/verifact.git
```

Step 2: Install Frontend Dependencies

```
npm install
```

Step 3: Install Backend Dependencies

```
pip install -r requirements.txt
```

Step 4: Start Backend Server

```
python app.py
```

Step 5: Run Frontend

```
npm start
```

Step 6:

Open the application in browser:

```
http://localhost:3000
```

4.4 End User Instructions

The VeriFact system is designed to be simple and user-friendly.

Steps to Use the System

Step 1

Launch the application in a web browser.

Step 2

Click **Upload** and select an image file.

Step 3

Preview the uploaded media.

Step 4

Click **Analyze** to begin deepfake detection.

Step 5

Wait for the AI model to process the media.

Step 6

View output:

- **Real Media** → Authentic content
- **Fake Media** → Manipulated content detected

The system also displays a confidence score.

Step 7

Click **Reset** to analyze another file.

Maintenance Activities

To maintain the system, the following activities are performed:

- Updating datasets with new deepfake samples
- Retraining models for improved accuracy
- Bug fixing and security updates
- Performance optimization
- Future model upgrades

Chapter 5: Results and Discussions

5.1 User Interface Representation of the Project

The user interface of **VeriFact** is designed to be simple, interactive and user-friendly, allowing users to upload media and detect deepfakes efficiently.

Main Interface Components

Home Page

The home page provides:

- Project title and introduction
- Deepfake detection purpose
- Navigation options
- Upload feature access

Media Upload Interface

This module allows users to:

- Upload image or video files
- Preview uploaded media
- Start analysis using the **Analyze** button

Result Display Interface

After processing, the system displays:

- Whether the media is **Real** or **Fake**
- Confidence score of prediction
- Additional detection insights (if available)

5.2 Brief Description of Various Modules of the System

The VeriFact system consists of the following major modules:

1. User Input Module

Accepts image or video uploads from users for analysis.

Functions:

- File selection
- File validation

- Media preview

2. Preprocessing Module

Prepares uploaded media for model analysis.

Functions:

- Face extraction
- Feature selection
- Image normalization

3. Deepfake Detection Module

Core AI module that analyzes media using deep learning algorithms.

Functions:

- Feature extraction
- Real/Fake classification
- Confidence prediction

4. Result Generation Module

Displays final detection results to users.

Outputs:

- Authentic Media
- Manipulated Media
- Confidence score

5. Reset/Reuse Module

Allows users to clear results and upload new files.

5.3 Snapshots of System with Brief Detail of Each

Snapshot 1: Home Page

Shows the landing page of VeriFact with navigation and introduction.

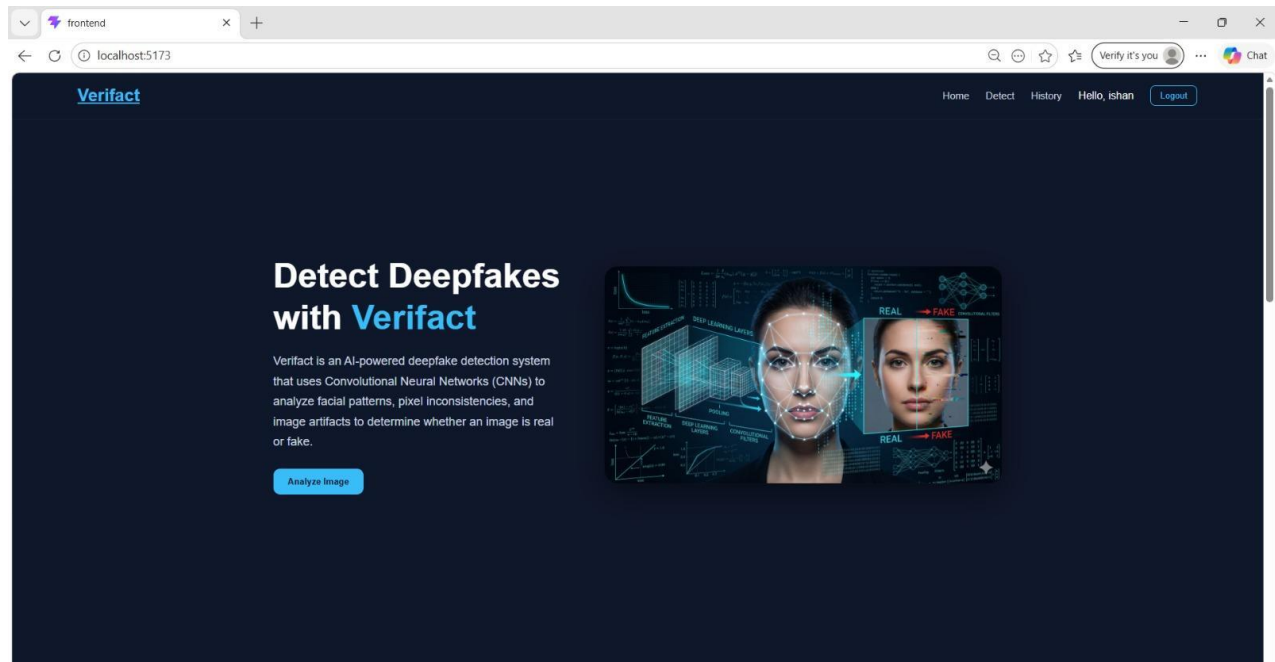


Fig 5.3.1

Snapshot 2: Upload Interface

Displays media upload section where users select image or video for detection.

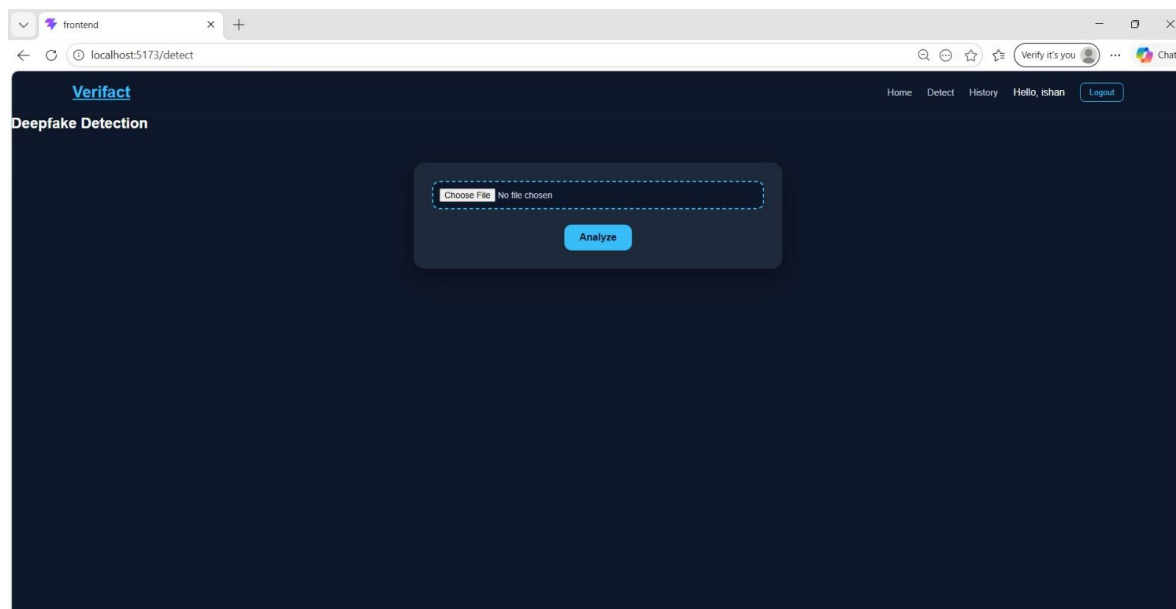


Fig 5.3.2

Snapshot 3: Media Preview Screen

Shows uploaded file preview before analysis.

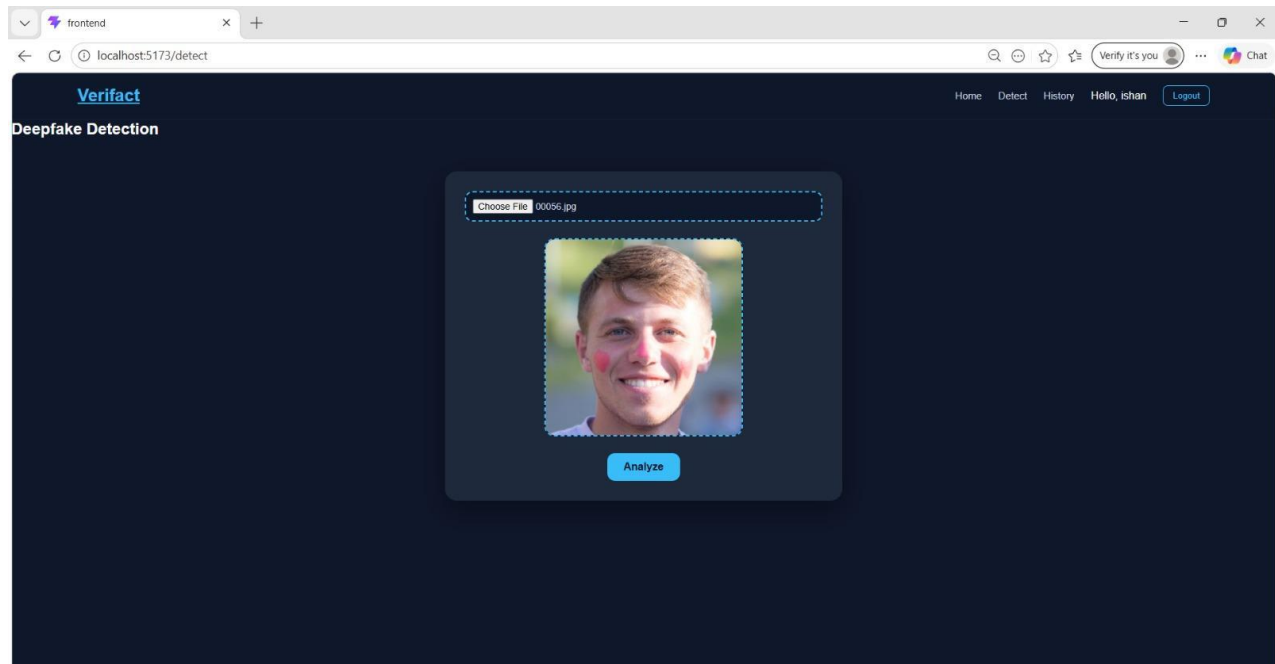


Fig 5.3.3

Snapshot 4: Detection Result Screen

Displays classification result as Real or Fake with confidence score.

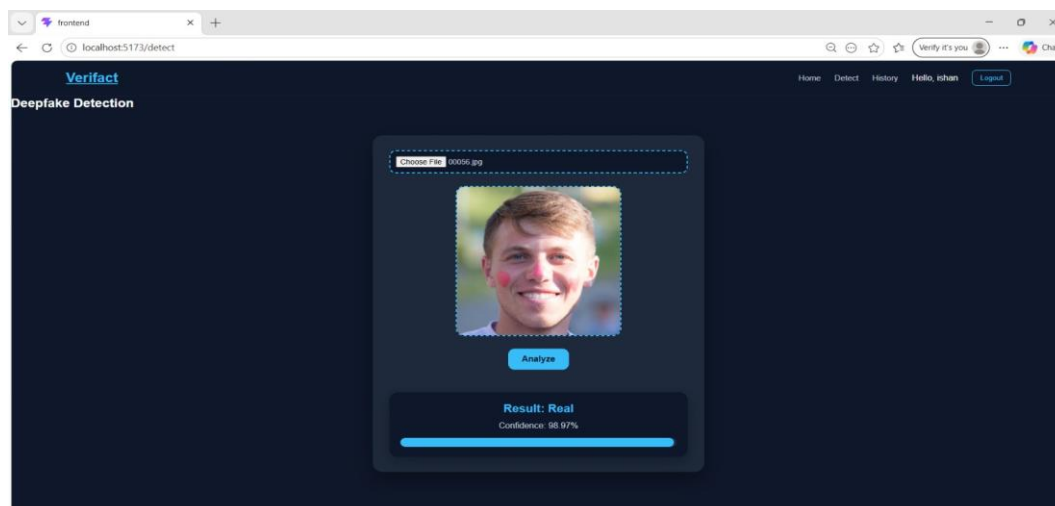


Fig 5.3.4

5.4 Back-End Representation (Database Used)

The backend of VeriFact manages data storage and communication between frontend and detection model.

Database Used

SQLite

Database stores:

- User information
- Uploaded media metadata
- Detection history
- Confidence scores
- System logs

Example Database Structure

Users Table

Stores registered user details.

Fields:

- User_ID
- Name
- Email
- Password

Detection Results Table

Stores prediction outputs.

Fields:

- Result_ID
- Prediction
- Created_at
- Username
- Image_path
- Confidence_Score

5.5 Snapshots of Database Tables with Brief Description

Snapshot 1: Users Table

Contains registered user information.

	id	username	email	password
1	1	sample	sample@gmail.com	script:...
2	2	ishan	Ishan@gmail.com	script:...

Fig 5.5.1

Snapshot 2: Detection Results Table

Stores classification outcomes and confidence scores.

	id	username	image_path	prediction	confidence	created_at
1	1	sample	uploads\ced7c481-2030-4984-90b9-5482...	Fake	0.00214281980879605	2026-04-22 20:19:49
2	2	sample	uploads\c5b6efdc-55b4-4163-a70f-...	Fake	0.0444686263799667	2026-04-22 20:20:09
3	3	sample	uploads\c1765ff4-75bf-4c74-81d5-6939...	Fake	0.00113165494985878	2026-04-22 20:20:14
4	4	sample	uploads\6b847254-f864-42e8-976d-...	Fake	0.00522467121481895	2026-04-22 20:20:19
5	5	sample	uploads\efceebe7-...	Fake	0.0329982228577137	2026-04-22 20:20:25
6	6	sample	uploads\c14a9281-4a98-4535-b03e-...	Fake	7.53331823943881e-06	2026-04-22 20:20:31
7	7	sample	uploads\3d937c36-2910-4f14-94d5-...	Fake	0.000575013866182417	2026-04-22 20:20:39
8	8	sample	uploads\b6327692-ff8b-4461-b2d6-...	Fake	8.77672355272807e-05	2026-04-22 20:20:46
9	9	sample	uploads\9a1ae642-66ac-4a54-8800-250f...	Real	0.999758541584015	2026-04-22 20:21:18
10	10	sample	uploads\77aa65b5-67d8-4955-be8a-...	Real	0.621195495128632	2026-04-22 20:23:17
11	11	sample	uploads\78a3d4fb-339f-4cc7-8291-5b4c...	Real	0.621195495128632	2026-04-22 20:23:34
12	12	ishan	uploads\85c3b278-effe-4d2b-...	Real	0.989698469638824	2026-04-23 06:07:29
13	13	ishan	uploads\4ef2ac3e-1dcc-4860-...	Real	0.989698469638824	2026-04-23 06:11:07

Fig 5.5.2

Chapter 6: Summary and Conclusions

6.1 Summary

The project **VeriFact: AI-Based Deepfake Detection System** was developed to identify manipulated digital media and help verify authenticity of images.

The system uses artificial intelligence and deep learning techniques to analyze uploaded media, detect forged content, and classify it as real or fake. The project integrates a user-friendly web interface with backend processing and machine learning models to provide an efficient detection mechanism.

Major functionalities achieved in the project include:

- Uploading image files for analysis
- Preprocessing and extracting important features
- Detecting deepfake content using AI models
- Displaying prediction results with confidence score
- Providing a simple and accessible web-based solution

The project successfully demonstrates the practical application of AI in addressing the growing problem of digital misinformation and synthetic media manipulation.

6.2 Conclusion

Deepfakes have become a major challenge in cybersecurity, media authenticity, and digital trust. This project presents an effective approach for detecting manipulated content using artificial intelligence. VeriFact provides a reliable framework for identifying forged media and can support applications in social media verification, journalism, cybersecurity, and digital forensics. The project met its objective of designing and implementing a deepfake detection system that is accurate, scalable, and user-friendly. Thus, the proposed system offers a promising solution to combat deepfake-based misinformation.

Chapter 7: Future Scope

Although the current system performs deepfake detection effectively, several improvements can be made in the future. The following enhancements can further improve the performance and usability of the proposed system:

7.1 Improved Model Accuracy

Advanced deep learning architectures can be implemented to enhance detection accuracy, especially for highly sophisticated and realistic deepfakes.

7.2 Real-Time Deepfake Detection

The system can be extended to support live or real-time detection for streaming videos and video conferencing applications.

7.3 Multimodal Deepfake Detection

Future versions of the system can incorporate multimodal analysis, including:

- Facial manipulation detection
- Voice cloning detection
- Lip-sync deepfake detection
- Audio-video inconsistency analysis

7.4 Browser Extension Integration

VERIFACT can be developed as a browser extension to allow users to verify the authenticity of online media directly while browsing.

7.5 Blockchain-Based Verification

Blockchain technology can be integrated to provide secure and tamper-proof verification of digital media authenticity.

7.6 Mobile Application Development

A mobile application version of the system can be developed to improve accessibility and usability across a wider range of users.

7.7 Larger Dataset Training

Training the model on larger and more diverse datasets can improve robustness and adaptability against evolving deepfake techniques.

APPENDIX

The appendix includes supplementary material related to the project **VERIFACT – AI-Based Deepfake Detection System**. It provides additional information that supports the main content of the report but is not included in the main chapters. This section helps in better understanding the implementation details, system behavior, and evaluation process.

Appendix A: Source Code Overview

The project is developed using a combination of frontend and backend components.

- **Frontend:** React-based interface for uploading and displaying results
- **Backend:** Python-based deep learning model for prediction
- **Model Implementation:** PyTorch-based neural network for deepfake classification
- **Image Processing:** PIL (Pillow) and Torchvision used for preprocessing

Appendix B: Sample Input and Output Results

Input Example:

- Uploaded File: Human face image
- Format: JPG / PNG

Output Example:

- Prediction: Real / Fake
- Confidence Score: 92.5% Fake / 7.5% Real

Appendix C: Test Cases

Test Case	Input	Expected Output	Result
TC-01	Real face image	Classified as Real	Pass
TC-02	Deepfake image	Classified as Fake	Pass
TC-03	Invalid file type	Error message	Pass
TC-04	Corrupted image	Rejected input	Pass

Table 7.1

Appendix D: User Manual

1. Open the application interface
2. Click on **Upload Image** button
3. Select an image or media file
4. Click on **Analyze**
5. View prediction result (Real/Fake) with confidence score
6. Use **Reset** to analyze another file

Appendix E: Dataset Details

The model is trained using publicly available deepfake datasets, including:

- FaceForensics++ dataset
- DeepFake Detection Challenge (DFDC) dataset

Dataset includes:

- Real face images/videos
- Manipulated (deepfake) media
- Balanced training and testing split

Appendix F: Model Architecture Overview

The system uses a deep learning-based classification model built using convolutional neural networks.

Pipeline:

- Input Image → Preprocessing → Feature Extraction → CNN Model → Classification Output

Key Components:

- Convolutional layers for feature extraction
- Pooling layers for dimensionality reduction
- Fully connected layers for classification
- Softmax/Sigmoid activation for final prediction

BIBLIOGRAPHY

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y., *Generative Adversarial Networks*, Advances in Neural Information Processing Systems (NeurIPS), 2014.
- TensorFlow Documentation, Google. Available: <https://www.tensorflow.org/>
- OpenCV Documentation, Open Source Computer Vision Library. Available: <https://opencv.org/>
- Rossler, A. et al., *FaceForensics++: Learning to Detect Manipulated Facial Images*, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.
- DeepFake Detection Challenge (DFDC) Dataset Documentation, Kaggle / Facebook Research Initiative, 2020.
- Articles and survey papers on deepfake detection using Deep Learning, Convolutional Neural Networks (CNNs), and Generative Adversarial Networks (GANs) from IEEE Xplore Digital Library and related journals in Artificial Intelligence and Digital Forensics.